

Урок 2. Корутины. Continuation

26 ноября 2019

В этом уроке разберем, что такое Continuation и как suspend функция приостанавливает код, не блокируя поток.

В прошлом уроке мы выяснили что Continuation помогает suspend функциям приостанавливать код, но не блокировать поток. Давайте разберем, как именно это происходит.

Подробно

Continuation

В качестве примера используем корутину:

```
1 launch {
2     val url = buildUrl()
3
4     download(url) //suspend function
5
6     toast("File is downloaded: " + url)
7 }
```

download - это suspend функция, которая загружает файл. Позже мы разберем, как создавать такие функции.

Пока же я напомним, что suspend функция download не заблокирует поток, в котором она будет вызвана. Потому что свою работу она будет выполнять в отдельном потоке. Но при этом она приостановит выполнение кода. И функция toast будет вызвана только после того, как download загрузит файл.

Сейчас мы подробно рассмотрим, как такое возможно.

Как вы знаете, Kotlin код будет при компиляции преобразован в Java классы. И корутина - не исключение. Более того, для нее будет создан отдельный класс. Именно в этом классе и содержится механизм приостановки корутины suspend функцией.

Т.е. то, что в Kotlin выглядит как магия корутин и suspend функций, под капотом (в Java) реализовано обычным кодом в обычном классе. Но разобравшись в коде этого класса - непросто. Поэтому мы начнем с очень упрощенной версии, из которой я убрал почти всю логику. По мере объяснения, я буду логику возвращать, получая код все более похожий на оригинал.

Итак, в результате преобразования кода корутины в Java будет создан Continuation класс. Его максимально упрощенная версия выглядит так:

```
1 class GeneratedContinuationClass extends SuspendLambda {
2     String url;
3
4     @Override
5     void invokeSuspend() {
6         url = buildUrl();
7
8         download(url); // suspend function
9
10        toast("File is downloaded: " + url);
11    }
12 }
```

В метод invokeSuspend переехал весь код, который корутина должна выполнить. Этот метод будет вызван при старте корутины.

Напомним, что мы обсуждаем, как магия Kotlin реализована под капотом, т.е. - Java код. Здесь уже нет никакой корутины или suspend функции. Они превратились в метод invokeSuspend и асинхронную функцию download (я продолжу называть ее suspend).

Основная задача Continuation - сделать так, чтобы toast был выполнен только после того, как функция download в фоновом потоке загрузит файл. Обычно в Java в таких случаях используется колбэк, но Continuation идет другим путем. Код в методе invokeSuspend делится свитчем на две части. И добавляется переменная label.

```
1 int label;
2
3 void invokeSuspend() {
4     switch (label) {
5         case 0: {
6             url = buildUrl();
7
8             download(url); // suspend function
9             return;
10        }
11        case 1: {
12            toast("File is downloaded: " + url);
13            return;
14        }
15    }
16 }
```

Точка разделения кода на две части - это suspend функция. Она и весь код перед ней попадет в первую часть. А весь код после нее - во вторую.

Теперь при вызове метода `invokeSuspend` от переменной `label` зависит, какая из двух частей будет выполнена. Если `label = 0`, то выполнится первая часть кода (`buildUrl` + `download`). А если `label = 1`, то вторая (`toast`). Это дает возможность отделить вызов `download` от `toast`.

Напомню, что первый раз метод `invokeSuspend` вызывается при старте корутины. При этом вызове отработает первая часть кода (в `case 0`) и запустится `suspend` функция, после чего метод `invokeSuspend` завершится (`return`). После того, как `suspend` функция закончит свою работу, надо сделать второй вызов метода `invokeSuspend`, чтобы отработала вторая часть кода, т.е. `toast`.

Но перед этим вторым вызовом надо поменять значение `label` с 0 на 1. Это может сделать первая часть кода перед тем, как вызвать `suspend` функцию.

```
1  int label;
2
3  void invokeSuspend() {
4      switch (label) {
5          case 0: {
6              url = buildUrl();
7              label = 1;
8
9              download(url); // suspend function
10             return;
11         }
12         case 1: {
13             toast("File is downloaded: " + url);
14             return;
15         }
16     }
17 }
```

А второй вызов метода `invokeSuspend` будет поручен `suspend` функции, т.к. этот вызов должен быть выполнен по ее завершении.

Когда `suspend` функция проходит преобразование из Kotlin в Java, она получает дополнительный входной параметр с типом `Continuation`. Это интерфейс. И класс, который мы сейчас разбираем, наследует этот интерфейс. Т.е. в `suspend` функцию передается текущий `Continuation` объект - `this`.

```
1  int label;
2
3  void invokeSuspend() {
4      switch (label) {
5          case 0: {
6              url = buildUrl();
7              label = 1;
8
9              download(url, this); // suspend function
10             return;
11         }
12         case 1: {
13             toast("File is downloaded: " + url);
14             return;
15         }
16     }
17 }
```

Когда `suspend` функция закончит загрузку файла, она возьмет `Continuation`, который ей передали и вызовет его `invokeSuspend` метод (подробно о том, как это происходит, мы поговорим в уроке про создание `suspend` функций). Значение `label` было установлено в 1 (еще до запуска `suspend` функции), поэтому `switch` идет во вторую ветку и метод `toast` будет выполнен.

Получается, что `Continuation` является колбэком для `suspend` функции. Если в корутине есть несколько `suspend` функций, то `Continuation` будет колбэком для всех них. Чуть дальше мы рассмотрим пример с двумя функциями.

Давайте еще раз посмотрим на всю цепочку вызовов, чтобы стало понятнее. Я записал видео-презентацию. Используйте паузу, если вам необходимо время, чтобы обдумать какой-либо слайд подробно.

Теперь вы примерно представляете себе схему, используемую корутинами и suspend функциями.

Возврат значения suspend функцией

suspend функция может возвращать какое-либо значение, как результат своей работы. Давайте посмотрим, как эта ситуация обрабатывается в Continuation. Кроме этого я добавлю в пример еще одну suspend функцию, чтобы наглядно показать, как switch разделит код в таком случае.

Перепишем ранее рассмотренный пример корутины:

```
1 launch {
2     val url = buildUrl()
3
4     val file = download(url) // suspend function
5
6     toast("File is downloaded: $url")
7
8     unzip(file) // suspend function
9
10    toast("File is unzipped")
11 }
```

Функция download теперь возвращает файл, который мы используем в другой suspend функции - unzip. После выполнения unzip нам надо показать сообщение.

Код будет разделен на три части:

- 1) download и все, что перед ней
- 2) unzip и все, что между ней и download
- 3) все, что после unzip

Точки разделения - это suspend функции.

Упрощенный код Continuation мог бы выглядеть примерно так:

```
1 File file;
2
3 void invokeSuspend() {
4     switch (label) {
5         case 0: {
6             url = buildUrl();
7             label = 1;
8
9             file = download(url, this); // suspend function
10            return;
11        }
12        case 1: {
13            toast("File is downloaded: " + url);
14            label = 2;
15
16            unzip(file, this); // suspend function
17            return;
18        }
19        case 2: {
20            toast("File is unzipped");
21            return;
22        }
23    }
24 }
```

Но тут есть проблема. Этот код не будет работать корректно. Обратите внимание на передачу файла между функциями download и unzip.

Мы пишем:

```
1 file = download(url, this);
```

Но download при запуске только начнет загрузку файла в фоновом потоке, но не вернет файл сразу как результат вызова. Поэтому такой путь не сработает. Нужен другой способ получить результат от suspend функции download.

Давайте вспомним, что suspend функция вызывает invokeSuspend по завершении своей работы. А если она делает этот вызов, то пусть заодно и передает туда результат своей работы, т.е. File. Для этого у invokeSuspend есть входной параметр типа Object:

```
1 void invokeSuspend(Object result) {
2     ...
3 }
```

Осталось этот Object привести к типу File. Continuation знает, что функция download вызовет метод invokeSuspend и передаст туда File, как результат своей работы. Во время этого вызова label уже будет равен 1, а значит будет вызван код в ветке case 1. А значит именно там надо делать приведение типа Object к типу File.

```

1 | File file;
2 |
3 | void invokeSuspend(Object result) {
4 |     switch (label) {
5 |         case 0: {
6 |             url = buildUrl();
7 |             label = 1;
8 |
9 |             download(url, this); // suspend function
10 |             return;
11 |         }
12 |         case 1: {
13 |             file = (File) result;
14 |             toast("File is downloaded: " + url);
15 |             label = 2;
16 |
17 |             unzip(file, this); // suspend function
18 |             return;
19 |         }
20 |         case 2: {
21 |             toast("File is unzipped");
22 |             return;
23 |         }
24 |     }
25 | }

```

Приводим result к типу File и используем полученный file в функции unzip. Таким образом мы получили результат работы suspend функции download в invokeSuspend.

Давайте немного усложним пример и добавим выходной параметр функции unzip:

```

1 | suspend fun unzip(path: File): Long

```

unzip возвращает размер распакованного содержимого.

Будем использовать это значение в сообщении, который показываем после выполнения unzip:

```

1 | launch {
2 |     val url = buildUrl()
3 |
4 |     val file = download(url) // suspend function
5 |
6 |     toast("File is downloaded: $url")
7 |
8 |     val size = unzip(file) // suspend function
9 |
10 |    toast("File is unzipped, size = $size")
11 | }

```

Смотрим Java код:

```

1 | File file;
2 | Long size;
3 |
4 | void invokeSuspend(Object result) {
5 |     switch (label) {
6 |         case 0: {
7 |             url = buildUrl();
8 |             label = 1;
9 |
10 |            download(url, this); // suspend function
11 |            return;
12 |        }
13 |        case 1: {
14 |            file = (File) result;
15 |            toast("File is downloaded: " + url);
16 |            label = 2;
17 |
18 |            unzip(file, this); // suspend function
19 |            return;
20 |        }
21 |        case 2: {
22 |            size = (Long) result
23 |            toast("File is unzipped, size = " + size);
24 |            return;
25 |        }
26 |    }
27 | }

```

Используется та же самая схема, что и с download. Функция unzip по завершении вызовет метод invokeSuspend и передаст туда результат своей работы типа Long. А в ветке case 2 результат будет приведен к типу Long и далее использован в сообщении.

Если подытожить, то получается следующая схема взаимодействия Continuation и suspend функций. Метод invokeSuspend вызывает suspend функции, передает туда Continuation и завершает свою работу. А suspend функции по завершении своей работы снова вызывают invokeSuspend и возвращают поток выполнения кода в блок кода, который в корутине шел после этой функции. При этом они передают туда же результаты своей работы.

Кратко

Рассмотрим корутину:

```
1 launch {
2     val url = buildUrl()
3
4     download(url) //suspend function
5
6     toast("File is downloaded")
7 }
```

download - это suspend функция, которая загружает файл. Она не заблокирует поток, в котором выполняется корутина, т.к. для своей работы использует фоновый поток. Но при этом функция toast будет вызвана только после того, как отработает download.

Чтобы понять, что такое Continuation, мы взглянем на Java класс, который получится в результате преобразования Kotlin кода корутины. Но сразу разбираться в конечном результате этого преобразования будет слишком сложно. Поэтому мы начнем с упрощенной версии и далее будем добавлять в нее код, делая его более похожим на реальный.

Continuation класс:

```
1 class GeneratedContinuationClass extends SuspendLambda {
2
3     int label;
4     String url;
5
6     void invokeSuspend() {
7         switch (label) {
8             case 0: {
9                 url = buildUrl();
10                label = 1;
11
12                download(url, this); // suspend function
13                return;
14            }
15            case 1: {
16                toast("File is downloaded: " + url);
17                return;
18            }
19        }
20    }
21 }
```

Обратите внимание на то, что код корутины переехал в отдельный класс. Далее мы будем оперировать объектом этого класса.

Нас интересует метод invokeSuspend, где и располагается код корутины. Этот метод будет вызван при старте корутины.

Основная задача Continuation - сделать так, чтобы код, расположенный после вызова функции download, был выполнен только когда метод download завершил работу. Для этого код делится свитчем на две части. И добавляется переменная label. Точка деления кода на две части - это suspend функция. Она и весь код перед ней идет в первую часть. А весь код после нее и до конца корутины идет во вторую часть. От значения переменной label зависит, какая из двух частей будет выполнена при вызове invokeSuspend.

Первый раз метод invokeSuspend вызывается при старте корутины. Он выполнит первую часть кода, поменяет значение label на 1 и вызовет suspend функцию. Второй раз метод invokeSuspend будет вызван уже из suspend функции, когда она завершит свою работу. Для этого suspend функция при преобразовании из Kotlin в Java получает дополнительный входной параметр с типом Continuation. Это интерфейс. И класс, который мы сейчас разбираем, наследует этот интерфейс. Т.е. в suspend функцию мы передаем текущий Continuation объект - this.

Когда suspend функция закончит загрузку файла, она возьмет Continuation, который ей передали и вызовет его invokeSuspend метод. label был установлен в 1 (еще до запуска suspend функции), поэтому switch пойдет во вторую ветку и метод toast будет выполнен.

Таким образом Continuation является колбэком для suspend функции. Если в корутине есть несколько suspend функций, то Continuation будет колбэком для всех них.

Возврат значения suspend функцией

suspend функция может возвращать какое-либо значение, как результат своей работы. Давайте посмотрим, как эта ситуация обрабатывается в Continuation. Кроме этого я добавлю в пример еще одну suspend функцию, чтобы наглядно показать, как switch разделит код в таком случае.

Перепишем ранее рассмотренный пример корутины:

```
1 launch {
2     val url = buildUrl()
3
4     val file = download(url) // suspend function
5
6     toast("File is downloaded: $url")
7
8     val size = unzip(file) // suspend function
9
10    toast("File is unzipped, size = $size")
11 }
```

Функция download теперь возвращает файл, который мы используем в еще одной suspend функции - unzip. А unzip в свою очередь возвращает размер распакованного содержимого. Будем использовать это значение в сообщении, который показываем после выполнения unzip.

При преобразовании в java, код будет разделен на три части:

- 1) download и все, что перед ней
- 2) unzip и все, что между ней и download
- 3) все, что после unzip

Точки разделения - это suspend функции.

Взглянем на полученный из этой корутины метод invokeSuspend. Он стал немного сложнее, т.к. теперь я не стал убирать из него логику получения значений от suspend функций.

```
1 File file;
2 Long size;
3
4 void invokeSuspend(Object result) {
5     switch (label) {
6         case 0: {
7             url = buildUrl();
8             label = 1;
9
10            download(url, this); // suspend function
11            return;
12        }
13        case 1: {
14            file = (File) result;
15            toast("File is downloaded: " + url);
16            label = 2;
17
18            unzip(file, this); // suspend function
19            return;
20        }
21        case 2: {
22            size = (Long) result
23            toast("File is unzipped, size = " + size);
24            return;
25        }
26    }
27 }
```

Как мы ранее уже обсуждали, suspend функция по завершении своей работы вызывает метод invokeSuspend. И именно сюда же она и передает результат своей работы. Для этого у invokeSuspend есть входной параметр типа Object.

Suspend функция download при вызове invokeSuspend передаст файл (File). А в ветке case 1 будет выполнено приведение типа Object к типу File.

Suspend функция unzip при вызове invokeSuspend передаст размер (Long). А в ветке case 2 будет выполнено приведение типа Object к типу Long.

